

JAVA INTERVIEW PREPARATION

CHAPTER 19

ARRAYLIST AND LIST INTERNALS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

ArrayList and List Internals: Contiguous Storage, Growth, Views, and Workload Fit

Senior / Lead Java Developer Reading Chapter

ArrayList is the default general-purpose List for good reason: it is compact, cache-friendly, and provides fast indexed access. Its apparent simplicity can hide costly insertion patterns, repeated growth, retained capacity, boxing overhead, and subtle view behavior. Senior-level selection begins with the workload rather than the interface name.

The Internal Model

An ArrayList stores element references in a contiguous Object array and maintains a logical size separately from array capacity.

```
elementData capacity = 8
index:  0    1    2    3    4    5    6    7
       [A] [B] [C] [D] [null][null][null][null]
logical size = 4
```

The list object does not contain the element objects themselves. It contains references. Adding a large object adds one reference to the backing array; the large object lives elsewhere on the heap.

`get(index)` checks the index and directly accesses the array slot, so it is constant time. Modern CPUs also benefit from the contiguous reference array when iteration proceeds in order.

Size Is Not Capacity

Size is the number of list elements exposed by the API. Capacity is the backing array length. Capacity may exceed size so future appends do not allocate on every call.

```
List<Order> orders = new ArrayList<>(10_000);
System.out.println(orders.size()); // 0, not 10,000
```

Initial capacity reserves reference slots; it does not create elements. Oversizing every list wastes heap, while undersizing a known large batch causes repeated array copies.

Growth and Amortized Appends

When an append would exceed capacity, ArrayList allocates a larger array and copies existing references. OpenJDK implementations commonly grow by roughly 50 percent, but the exact growth formula is not an API contract.

```
before append: [A B C D] capacity 4
allocate:      [A B C D _ _] larger array
append:        [A B C D E _]
old array becomes garbage
```

One growth operation is $O(n)$, but a sequence of appends is amortized $O(1)$: expensive copies are spread across many cheap appends. Amortized does not mean every add is constant time. A resize on a latency-sensitive request can still be visible.

Use `ensureCapacity` before a known bulk load. Use `trimToSize` only with evidence; it copies the array and a later append may immediately grow it again.

Insertion and Removal Shift Elements

Adding at the end is normally cheap. Adding near the beginning shifts the suffix to the right. Removing by index shifts the suffix left and clears the now-unused final slot so the removed object can be collected.

```
insert X at index 1  
[A B C D _] -> [A _ B C D] -> [A X B C D]
```

The cost is $O(n)$ in the shifted portion. Repeatedly removing index zero turns a linear traversal into quadratic behavior.

```
while (!list.isEmpty()) {  
    process(list.remove(0)); // repeated full shifts  
}
```

Use `ArrayDeque` for FIFO queue behavior. Choose a collection whose structural operations match the algorithm.

The remove Overload Trap

`List` has both `remove(int index)` and `remove(Object value)`. With boxed integers, overload selection can surprise developers.

```
List<Integer> numbers = new ArrayList<>(List.of(10, 20, 30));  
numbers.remove(1); // removes element at index 1: 20  
numbers.remove(Integer.valueOf(10)); // removes the value 10
```

This is a compile-time overload decision, not an `ArrayList` bug. Be explicit when the element type is `Integer`.

Search Uses Equality

`contains`, `indexOf`, and `remove(Object)` scan elements and compare using equality. They are $O(n)$, and expensive equals implementations multiply that cost.

If code repeatedly asks whether an identifier is present, a `HashSet` may represent the access pattern better. Maintaining both a list and a set can be appropriate when order and fast membership are both required, but every mutation must keep them consistent.

Iterators and Structural Modification

`ArrayList` iterators are fail-fast on a best-effort basis. Structural modification outside the iterator can cause `ConcurrentModificationException`.

```
Iterator<Order> iterator = orders.iterator();  
while (iterator.hasNext()) {  
    if (iterator.next().isCancelled()) {  
        iterator.remove(); // supported iterator mutation  
    }  
}
```

Enhanced for-loops use iterators. Prefer `removeIf` for simple filtering, or build a new result when transformation is clearer. `ConcurrentModificationException` is not a concurrency control mechanism.

subList Is a Backed View

`subList` returns a view over a range of the original list, not an independent copy.

```
List<String> all = new ArrayList<>(List.of("A", "B", "C", "D"));
List<String> middle = all.subList(1, 3);
middle.set(0, "X");

System.out.println(all); // [A, X, C, D]
```

Structural modification of the backing list outside the view invalidates the view's assumptions and later operations may throw `ConcurrentModificationException`. A small long-lived `subList` can also retain the entire backing list through its parent relationship.

When an independent result is required, copy it:

```
List<String> page = new ArrayList<>(all.subList(from, to));
```

list.toArray and Generic Arrays

Because generics are erased and arrays are reified, conversions deserve care.

```
Order[] array = orders.toArray(Order[]::new);
```

The generator form clearly creates the correct runtime array type. An `Object[]` cast to `Order[]` is not valid merely because its current elements happen to be orders.

Memory and Boxing

An `ArrayList<Integer>` stores references to `Integer` objects, not packed primitive `int` values. Large numeric datasets may pay for wrappers, references, allocation, and poorer locality.

```
ArrayList<Integer>: [ref][ref][ref] -> separate Integer objects
int[]:               [int][int][int] packed values
```

For computation-heavy primitive data, arrays or a specialized primitive collection can dramatically reduce memory and improve locality. Measure before adding a third-party dependency, but understand the representation cost.

Clearing a list nulls its active references but normally retains backing-array capacity. Reusing a large list can avoid allocation; retaining it forever after a one-time spike can waste memory. Lifecycle determines the right choice.

ArrayList Versus LinkedList

`LinkedList` offers constant-time insertion or removal only after the relevant node is already available through an iterator. Finding index `n` remains $O(n)$. Each node stores element and link references, increasing allocation and reducing locality.

For most application lists, `ArrayList` is faster and smaller. `LinkedList` can fit algorithms that perform frequent iterator-local insertion/removal and do not need indexed access, but `ArrayDeque` is usually better for stack and queue operations.

```
ArrayList: compact references, fast index, shifting on middle edits
LinkedList: per-node allocation, traversal for index, cheap known-node unlink
ArrayDeque: circular array, efficient operations at both ends
```

CopyOnWriteArrayList

CopyOnWriteArrayList creates a new backing array for every structural update. Reads and iteration are simple and snapshot-like, so it fits small collections with rare writes and frequent traversal, such as listener registrations.

It is a poor choice for write-heavy data. An add to a million-element list copies a million references. Its iterator does not reflect later changes and does not support iterator removal.

Thread Safety and Publication

ArrayList is not thread-safe. Concurrent mutation can produce races, lost changes, stale reads, or inconsistent iteration. A safely published list that is never mutated can be read concurrently, but immutability must include the intended element state.

Options include external locking, synchronized wrappers with correct iteration locking, CopyOnWriteArrayList for read-mostly workloads, concurrent queues for producer-consumer flows, or immutable snapshots replaced atomically.

Lazy Allocation and Java 21 Growth Mechanics

A newly constructed default ArrayList can share an empty sentinel array until the first element is added. The first growth then creates storage with the usual default capacity. This avoids allocating ten reference slots for every empty list that may never receive an element.

```
new ArrayList<>()
list object -> shared empty sentinel
                |
                | first add
                v
            private Object[10]
```

For a non-empty backing array, the Java 21 implementation asks for a new length using the larger of the minimum growth required and a preferred growth of half the old capacity. In ordinary cases this produces roughly 1.5 times capacity. The API promises amortized constant-time append, not the exact multiplier, so application correctness must not depend on 1.5.

```
// Conceptual, not public ArrayList source API
preferredGrowth = oldCapacity >> 1;
newCapacity = enoughFor(size + 1, preferredGrowth);
elementData = Arrays.copyOf(elementData, newCapacity);
```

`ensureCapacity` is useful when a batch size is genuinely known. Calling it with exaggerated guesses moves the cost from copying to retained heap and can harm iteration locality. Capacity planning should use observed cardinality distributions, not only the maximum ever seen.

What Shifting Does to the Array

Middle insertion uses an array-copy operation to move a contiguous suffix. Removal performs the reverse shift and then nulls the now-unused last active slot.

```
remove index 1 from [A B C D _ _]

before: [A B C D _ _]
copy:   [A C D D _ _]
clear:  [A C D _ _ _]   size becomes 3
```

Nulling is essential. Without it, the backing array would retain a reference to `D` beyond logical size and delay garbage collection. The JVM can optimize bulk reference copies, but memory bandwidth and garbage-collector barriers still make large shifts observable.

One middle edit may be acceptable; an algorithm performing thousands of edits near the front is the problem. Complexity analysis should count total movement across the whole workflow rather than labeling one call in isolation.

Bulk Filtering and Compaction

Repeatedly removing individual elements can cause repeated suffix shifts. `removeIf` and internal bulk-removal strategies can compact survivors in a smaller number of passes.

```
orders.removeIf(Order::isExpired);
```

Conceptually, the algorithm identifies removals, moves survivors toward the front, and clears the unused tail:

```
input:      [keep drop keep drop keep]
compact:    [keep keep keep drop keep]
clear tail: [keep keep keep _ _]
```

The predicate must not structurally mutate the same list. For complex transformations or when the original must remain unchanged, collecting a new list often gives clearer ownership and failure behavior.

RandomAccess Is an Algorithmic Signal

`ArrayList` implements the `RandomAccess` marker interface. It tells generic algorithms that indexed access is expected to be efficient. `LinkedList` does not.

```
static long sum(List<Integer> values) {
    if (values instanceof RandomAccess) {
        long total = 0;
        for (int i = 0; i < values.size(); i++) total += values.get(i);
        return total;
    }
    long total = 0;
    for (int value : values) total += value;
    return total;
}
```

In ordinary application code, enhanced iteration is usually clear for both. The marker matters when implementing reusable algorithms where repeated indexed access could turn a linear operation on `ArrayList` into quadratic traversal on `LinkedList`.

List Equality, Hash Codes, and Mutable Elements

List equality is order-sensitive and element-based. Two lists are equal when they have the same size and corresponding elements are equal in the same order. Their implementation classes need not match.

```
new ArrayList<>(List.of("A", "B"))
    .equals(new LinkedList<>(List.of("A", "B"))); // true
```

This contract allows lists to work as values independent of implementation choice. Using a mutable list as a `HashMap` key is dangerous: changing an element, order, or size changes the list's hash code and can make the entry unreachable by lookup. An unmodifiable wrapper prevents structural methods through that reference but does not necessarily freeze mutable elements or other aliases to the backing list.

Fixed-Size, Unmodifiable, and Immutable-Looking Lists

Several APIs create lists with different mutation and ownership semantics:

API	structural mutation	backing relationship
<code>new ArrayList<>(source)</code>	allowed	independent element refs
<code>Arrays.asList(array)</code>	add/remove forbidden	backed by supplied array
<code>Collections.unmodifiableList(x)</code>	forbidden via wrapper	live view of x
<code>List.of(...)</code>	forbidden	compact unmodifiable value
<code>List.copyOf(source)</code>	forbidden	unmodifiable result
<code>subList(...)</code>	allowed by contract	view of parent range

`Arrays.asList` permits `set` because replacing an array slot does not change size. An unmodifiable wrapper reflects modifications made through another reference to its backing list. `List.copyOf` creates an unmodifiable result and rejects null elements, but it is a shallow copy: mutable element objects remain mutable.

Choose based on ownership. “The caller cannot call `add`” is weaker than “no part of the system can mutate the representation.”

SequencedCollection Operations in Java 21

In Java 21, `List` participates in the sequenced-collection model and exposes `first/last` and `reversed-view` operations through its hierarchy. The common API does not change `ArrayList`'s physical costs.

```
list.addFirst(value); // valid API, but ArrayList shifts existing elements
E first = list.getFirst();
List<E> reversed = list.reversed(); // reverse-ordered view
```

An operation being available on the interface does not make it equally efficient on every implementation. `addFirst` is natural for a deque but $O(n)$ for `ArrayList`. `reversed()` is a view, so understand backing and mutation behavior rather than assuming a detached reversed copy.

Sorting, Spliterators, and Streams

`ArrayList` sorting rearranges references in the backing storage according to the comparator. Comparator correctness matters: it should be consistent enough to define an ordering and must not mutate the list during comparison.

Streams do not change collection complexity. Filtering an `ArrayList` stream remains a traversal; repeated `list.contains` inside another stream can still be quadratic. `ArrayList`'s spliterator can divide an index range efficiently for parallel processing, but parallelism helps only when data size and per-element work justify coordination overhead.

Do not mutate a non-concurrent `ArrayList` from parallel stream workers:

```
List<Result> results = new ArrayList<>();
inputs.parallelStream().forEach(x -> results.add(transform(x))); // race
```

Prefer a proper collector or keep the pipeline sequential. Even a synchronized destination can introduce contention and ordering surprises.

API Ownership and Defensive Copies

Returning an internal mutable list gives callers an alias to your state:

```
final class OrderBatch {
    private final List<Order> orders;

    OrderBatch(List<Order> orders) {
        this.orders = List.copyOf(orders);
    }

    List<Order> orders() {
        return orders;
    }
}
```

This protects list structure but not mutable `Order` objects. Decide whether domain elements are immutable, require copying, or are intentionally shared. Defensive-copy policy should be consistent at construction and access; copying only on return still allows the constructor's original list to mutate internal state.

For large collections, copying has real cost. Alternatives include immutable domain aggregates, ownership transfer, persistent collections, pagination, or read-only query interfaces. Performance does not justify undocumented aliasing.

A Strong Interview Explanation

Explain `ArrayList` as a logical size over a contiguous `Object[]`. Indexed access is constant time, append is amortized constant time, and growth occasionally allocates a larger array and copies all references. Middle insertion and removal are linear because a suffix shifts; repeated front removal therefore becomes quadratic.

Then cover the traps that distinguish a senior answer: capacity is not size, `subList` and reversed lists are backed views, fail-fast is not thread safety, boxed primitives have representation cost, and `remove(int)` differs from `remove(Object)`. Compare `ArrayList` with `ArrayDeque`, `LinkedList`, and `CopyOnWriteArrayList` by workload. Finish with ownership: unmodifiable structure, immutable elements, and defensive copying are separate decisions.

Production Perspective

Common incidents include repeated `remove(0)`, accidental quadratic contains loops, a `subList` retaining a huge dataset, a large backing array retained after a traffic spike, and boxed primitives multiplying heap usage. Another pattern is returning a mutable internal `ArrayList` from an API, allowing callers to break invariants.

Use allocation profiles, heap histograms, retained-size analysis, CPU profiles around arraycopy or equals, and realistic collection cardinalities. Complexity describes scaling; profiling shows whether the scaling matters in this service.

Interview-Ready Summary

`ArrayList` is a resizable `Object` array with separate size and capacity. Indexed access is $O(1)$, append is amortized $O(1)$, and middle insertion or removal is $O(n)$ because references shift. Growth requires allocation and copying.

`subList` is a backed view, fail-fast iteration is diagnostic, and `Integer` removal has an overload trap. `ArrayList` usually outperforms `LinkedList` because of locality and lower allocation. Choose `ArrayDeque` for queues, `CopyOnWriteArrayList` for small read-mostly collections, and primitive arrays or specialized collections when boxing dominates.

Revision Notes

- ArrayList stores references in a contiguous Object array; size and capacity are different.
- Initial capacity does not create elements, and default empty lists may allocate lazily.
- `get` is $O(1)$; append is amortized $O(1)$; growth copies $O(n)$ references.
- Java 21 prefers roughly half-capacity growth, but the multiplier is not API contract.
- Middle insertion and removal shift a suffix.
- Removal clears the unused tail; bulk filtering can avoid repeated shifts.
- Repeated `remove(0)` is usually the wrong queue design.
- `remove(int)` and `remove(Object)` are distinct overloads.
- `contains` and `indexOf` are linear equality searches.
- Iterators are fail-fast on a best-effort basis.
- `subList` is backed; copy when independent lifetime is required.
- `ArrayList<Integer>` stores boxed objects and references.
- `clear` normally retains backing capacity.
- ArrayList usually beats LinkedList; ArrayDeque normally fits stacks and queues better.
- `CopyOnWriteArrayList` favors many reads and very few writes.
- ArrayList requires immutability, external synchronization, or another design for concurrency.
- Profile array copying, equality scans, boxing, and retained capacity in production.
- `RandomAccess` signals that indexed algorithms are expected to be efficient.
- List equality is order-sensitive and independent of implementation class.
- `Arrays.asList`, wrappers, `List.copyOf`, and `subList` have different ownership semantics.
- Java 21 sequenced operations do not change ArrayList's physical complexity.
- Never mutate a shared ArrayList from parallel stream workers.
- Defensive copying of the list does not make mutable elements immutable.